Doc. no.:	P0849R3
Date:	2020-2-29
Audience:	EWG, LWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

auto(x): *decay-copy* in the language

Changes Since R2

- dropped `decltype(auto)(x)` in comply with EWG's opinion

Changes Since R1

- propose `decltype(auto)(x)` as well

Changes Since R0

- updated examples
- discussed `decltype(auto)(x)`
- added library wording

Introduction

This paper proposes `auto(x)` and `auto{x}` for casting `x` into a prvalue value as if passing `x` as a function argument by value. The functionality is realized as the *`decay-copy`* function in the standard for exposition only.

Motivation

Obtaining a prvalue copy is necessary

A generic way to obtain a copy of an object in C++ is `auto a = x;` but such a copy is an lvalue. We could often convey the purpose in code more accurately if we can obtain a copy as a prvalue. In the following example, let `Container` be a concept,

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), auto(x.front()));
}
```

If we wrote

```
void pop_front_alike(Container auto& x) {
    auto a = x.front();
    std::erase(x.begin(), x.end(), a);
}
```

, questions arise – why this is not equivalent to

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), x.front());
}
```

The problem is, the statement to obtain an lvalue copy is a declaration:

```
    auto a = x.front();
```

Its primary purpose is to declare a variable while being a copy is a property of the declaration. In contrast, the expression to obtain an rvalue copy is a clear command to perform a copy:

```
    auto(x.front())
```

One might argue that the above is indifferent from

```
    T(x.front())
```

However, there are plenty of situations that the `T` is nontrivial to get. We probably don't want to write the original example as

```
void pop_front_alike(Container auto& x) {
    using T = std::decay_t<decltype(x.front())>;
    std::erase(x.begin(), x.end(), T(x.front()));
}
```

Obtaining a prvalue copy with `auto(x)` works always

In standard library specification, we use the following exposition only function to fulfill the role of `auto(x)`:

```
template<class T>
constexpr decay_t<T> decay_copy(T&& v) noexcept(
    is_nothrow_convertible_v<T, decay_t<T>>) {
    return std::forward<T>(v);
}
```

This definition involves templates, dependent `constexpr`, forwarding reference, `noexcept`, and two traits, and still has caveats if people want to use it in practice. An obvious issue is that `decay_copy(x.front())` creates a copy of `x.front()` even if `x.front()` is already a prvalue (thus, already a copy).

There is a less obvious issue which needs a minimal reproduce:

```
class A {
    int x;

public:
    A();

    auto run() {
        f(A(*this));           // ok
        f(auto(*this));        // ok as proposed
        f(decay_copy(*this));  // ill-formed
    }

protected:
    A(const A&);
};
```

The problem is that `decay_copy` is nobody's friend. We can use `A` directly in this specific example. Still, in a more general setting, where a type `A` has access to a set of type `T`'s private or protected copy/move constructors, *`decay-copy`* an object of `T` fails inside `A`'s class scope, but `auto(x)` continues to work.

Discussion

`auto(x)` is a missing piece

Replacing the `char` in `char('a')` with `auto`, we obtain `auto('a')`, which is a function-style cast. Such a formula also supports *injected-class-names* and class template argument deduction in C++17. Introducing `auto(x)` and `auto{x}` significantly improves the language consistency:

variable definition	function-style cast	new expression
auto v(x);	auto(x)	new auto(x)
auto v{x};	auto{x}	new auto{x}
ClassTemplate v(x);	ClassTemplate(x)	new ClassTemplate(x)
ClassTemplate v{x};	ClassTemplate{x}	new ClassTemplate{x}

** *The type of `x` is a specialization of `ClassTemplate`.*

With this proposal, all the cells in the table copy construct form `x` (given CTAD's default behavior) to obtain lvalues, prvalues, and pointers to objects, categorized by their columns. Defining `auto(x)` as a library[1] facility loses orthogonality.

Introducing `auto(x)` into the language even improves the library consistency:

type function style	expression style
void_t<decltype(*expr*)>	decltype(void(*expr*))
decay_t<decltype(*expr*)>	decltype(auto(*expr*))

Do we also miss `decltype(auto){x}`?

`decltype(auto){arg}` can forward `arg` without computing `arg`'s type. It is equivalent to `static_cast<decltype(arg)>(arg)`. If `arg` is a variable of type `T&&`, `arg` is an lvalue but `static_cast<T&&>(arg)` is an xvalue.

EWG discussed this idea, disliked its expert-friendly nature, and concluded that adding this facility would cause the teaching effort to add up.

Implementation

Try it out: Godbolt
(https://godbolt.miator.net/#z:OYLghAFBqd5QCxAYwPYBMCmBRdBLAF1QCcAaPECAKxAEZSAbAQwDtRkBSAJgCFufSAZ1QBXYskwgA5NwDMeFsgYisAag6yAwi0wB3Ddg4AGAILGTBTAFsADs0vqtBAJ43MLJlcyqAKgfMAbqh46KoAZhB%2BXABs3NEAlBp8puYKBKpWTAoQiSkA7MkmqsWqyAhMxKpMHACsfDUAIo5N3FwseBKtSeYlqmmlqCyCBHGqlRpNACxc3aa9EUwiRBBM8YmyhfMrS6gcBcR7DeuFPSWLywAe8c2qYFJMd7NFJSKCCsCq6ROqWEoubttlqtjuZQVx5GFVEZTsUFjsciC5iU4UQ9nw8kcnltzqgIFxSLI1IjkYDdgV6DMMYizEjijpdFUdmjDklejDVPTGUDqb1OTi0ZSGsS6XoubiBYceSU%2BfD8YThRzRfyCvjJiyNmDZO58GFQRipPFGNIalJSCxpEZTahpJp%2BPxVMIxBJ1ODaKaCBaDYaANYgGpGI1SSam81SS2ka1SU2CEABj1hg2kOCwJBoWx4BiYMgUCBpmwZrMgKzOJSsYCkMIZyzEGMQABGntIdYUFWc0jdpDTXhYBAA8iwGG2E6QsJk2JnG/hiJhkAQ8AFMDHh5gLjOlpIpB20pgGI2CMQ8FZPYbmGwUHbeIw8HWY5BDagbHPBkuALQAdSYDAYqhfw3QE04Xh%2BC4Jho1EcRJFoE9jRDRtI2LUs2FUCBcEIEgXVkehVE0VB00zcZXWuW0gN4d1jx9P0A13INYOHSNo1jUh40tNZAy4Wjw3opjyNIBcazwQYQEmIA)

Wording

The wording is relative to N4849.

Modify 7.6.1.3 [expr.type.conv]/1 as indicated:

> A *simple-type-specifier* (9.2.8.2) or *typename-specifier* (13.8) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.4.1.8) for the remainder of this section. Otherwise, if the type is `auto`, it is replaced by the type deduced for the variable `x` in the invented declaration (9.2.8.5):

```
auto x init;
```

> , where *init* is the initializer.

Modify 9.2.8.5 [dcl.spec.auto]/5 as indicated:

> A placeholder type can also be used in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (7.6.2.7) and as a *decl-specifier* of the *parameter-declaration*'s *decl-specifier-seq* in a *template-parameter* (13.2). The `auto` *type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation) (7.6.1.3).

Remove the first entity from 16.4.2.1 [expos.only.func]/2:

~~template<class T> constexpr decay_t<T> *decay-copy*(T&& v)~~
~~ noexcept(is_nothrow_convertible_v<T, decay_t<T>>) // *exposition only*~~
~~{ return std::forward<T>(v); }~~

Search and replace “~~calls to *decay-copy* being evaluated in~~” (the thread/the current thread) with “where the values produced by `auto` are materialized in”.

Search and replace “~~*decay-copy*~~” with “`auto`”.

Acknowledgments

Thank Alisdair Meredith, Arthur O’Dwyer, and Billy O’Neal for providing examples and feedback for this paper. Thank James Touton for presenting the paper and bringing it forward.

References

1. Krügler, Daniel. P0758R0 *Implicit conversion traits and utility functions*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html) ↩